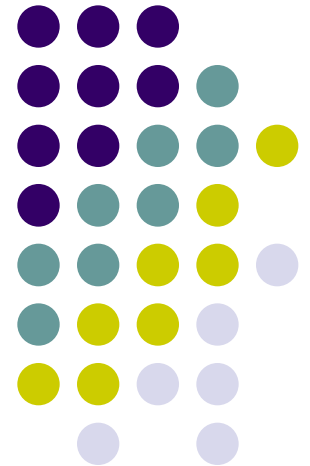
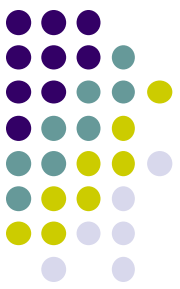


数据仓库的设计

Design of Data Warehouse

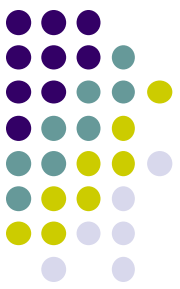
Software Institute, Nanjing University
Bei Jia





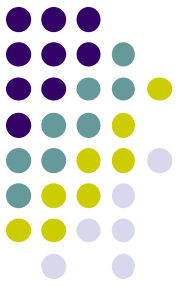
数据仓库设计的由来（1/2）

- 在事务型数据处理中需要作数据库设计，而在分析型数据处理中则需作数据仓库设计，这两者在原理上是一致的
- 因此，数据库设计中的很多设计思想与方法都可在数据仓库中得到应用
- 但是由于事务型与分析型的数据处理的不一致，两者在设计中的很多方面也存在着差别



数据仓库设计的由来（2/2）

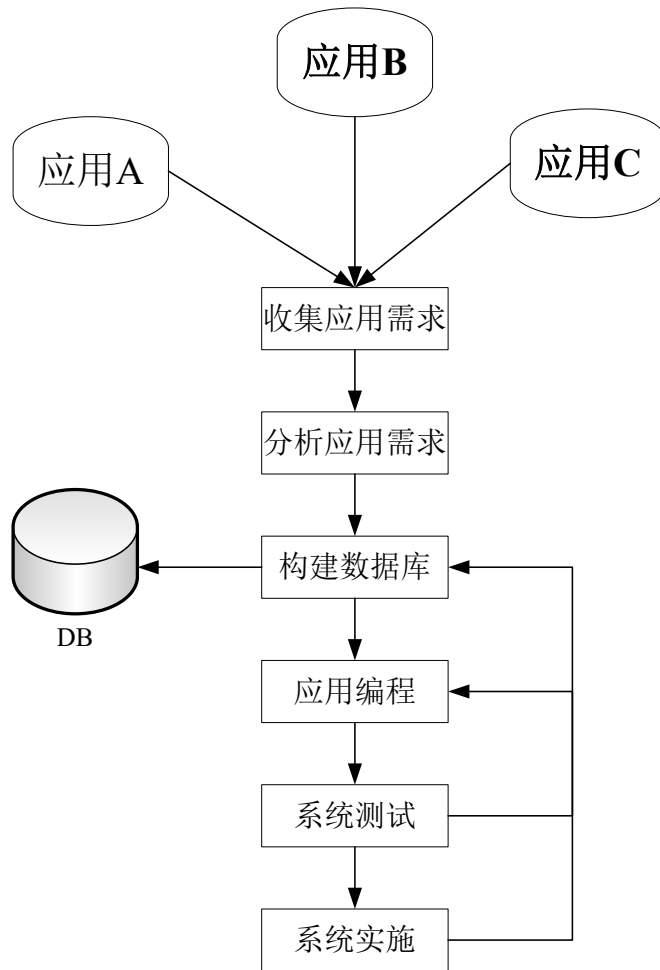
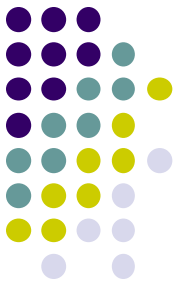
- 面向OLTP的数据库设计有着明确的应用需求，严格遵循系统生命周期的阶段划分，每个阶段都规定有明确的任务，上一阶段确定的任务完成后，产生一定格式的文档交给下一阶段
- 建造数据仓库重要有两部分
 - 与操作型系统接口的设计（ETL）
 - 数据仓库本身的设计
- 数据仓库的需求只有在已装载部分数据并已开始使用时才能弄清楚。
- 无法使用传统的需求驱动的方法进行



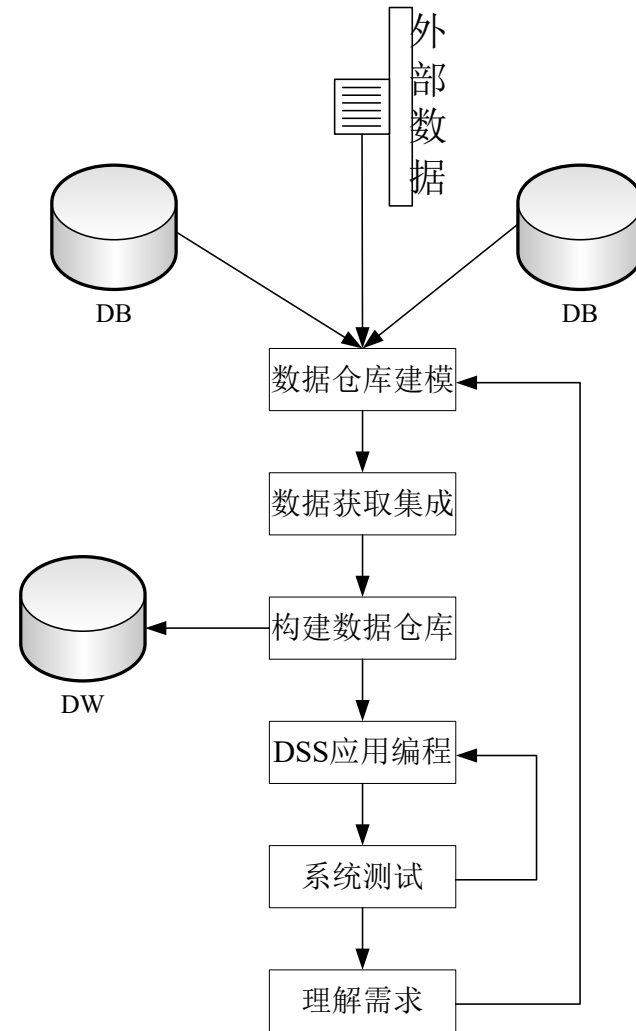
操作型数据库 vs. 数据仓库

- 面向的处理类型不同
 - 面向应用 vs. 面向分析
- 面向的需求不同
 - 确定的应用需求 vs. 不确定的分析需求
- 系统设计目标不同
 - 事务处理性能 vs. 全局一致的数据环境
- 数据来源或系统的输入不同
 - 事务相关数据 vs. 多种多样
- 系统设计的方法和步骤不同
 - SDLC（系统生命周期） vs. CLDS

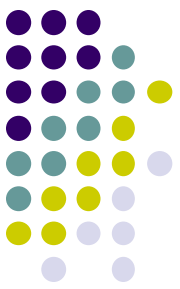
SDLC vs. CLDS



SDLC方法

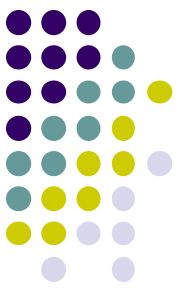


CLDS方法



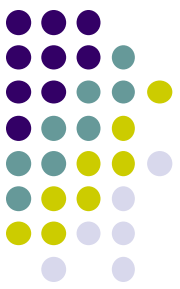
数据仓库设计的原则

- 数据仓库是面向主题的、集成的、非易失、时变的，这些特点决定了数据仓库的系统设计不能采用同开发传统的OLTP数据库一样的设计方法，其设计过程必须遵循下述三条原则：
 - 面向主题原则
 - 数据驱动原则
 - 原型法设计原则



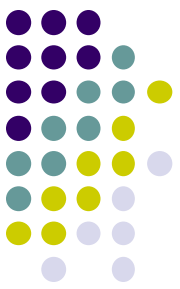
设计原则 1 - 面向主题原则（1/2）

- 建立数据仓库的目的
 - 构建数据仓库的目的是面向企业的管理人员，为经营管理提供决策支持信息。因此，数据仓库的组织设计必须以用户决策的需要来确定，即从用户决策的主观需求（主题）开始
- 数据仓库中数据的组织方法
 - 为了进行数据分析首先要有分析的主题，以主题为起始点，进行相关数据的设计，最终建立起一个面向主题的分析型环境
 - 在数据库设计中则是以客体为起始点，即以客观操作需求为设计依据



设计原则 1 - 面向主题原则 (2/2)

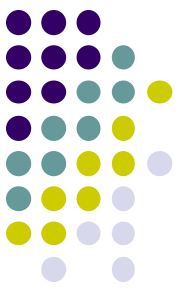
- 例如：“商品销售”主题
 - 建立目的
 - 管理人员能够在适当的时候，订购适当的商品，并把它们分发到适当的商店中去销售，以提高商品的销售总金额
 - 需要执行的分析操作
 - 分析什么样的商品，在什么样的时间和商店内畅销
 - 即：分析商品的销售额与商品类型、销售时间及商店位置之间的变化关系
 - 管理人员将据此决定他们的经营策略



设计原则 2 - 数据驱动原则（1/3）

- 数据的来源

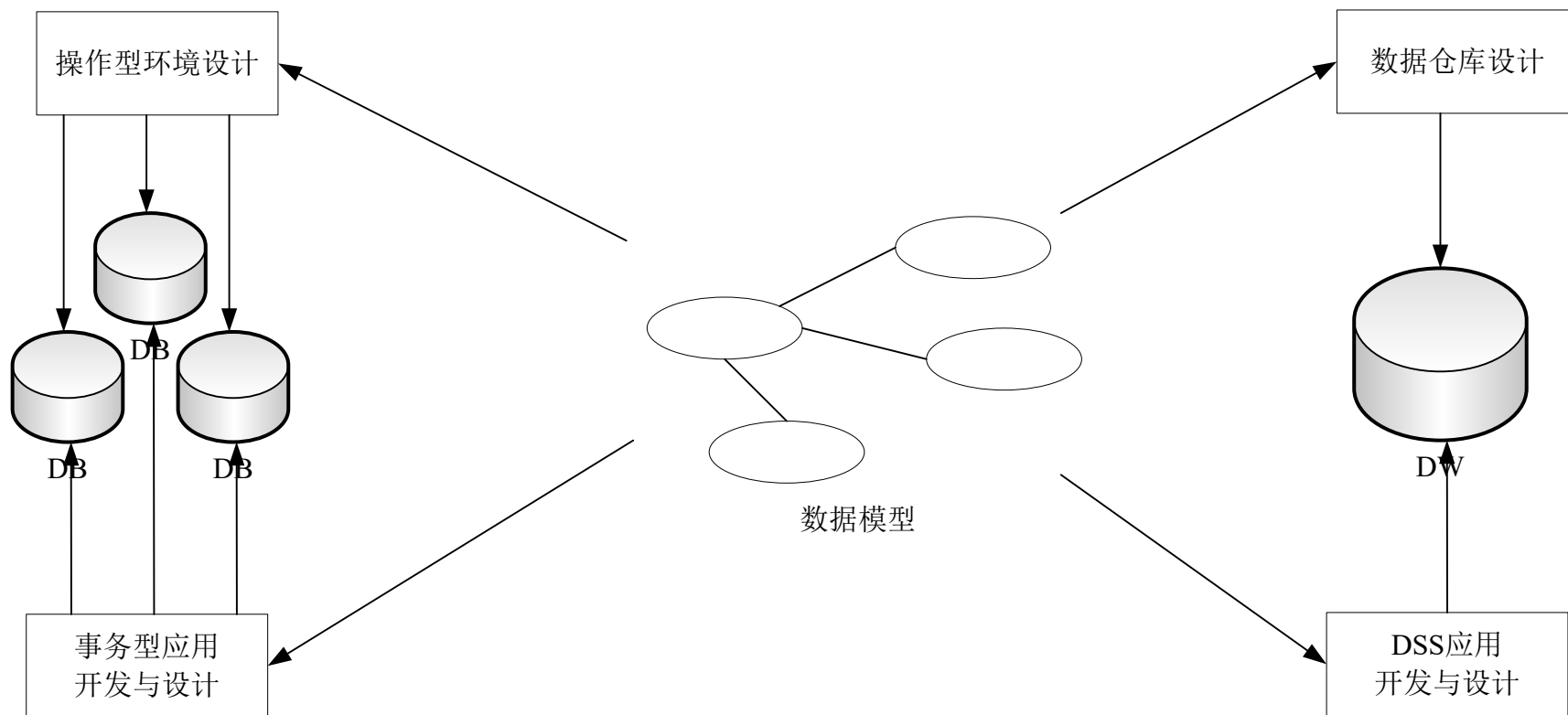
- 数据仓库是在现存数据库系统基础上进行开发的，它着眼于有效地提取、综合、集成和挖掘已有数据库中的数据资源，服务于企业高层领导管理决策分析的需要
- 因此，数据仓库中的数据必须是从已有的数据源中抽取而来，是已经存在的数据或对已经存在的数据进行加工处理而获得

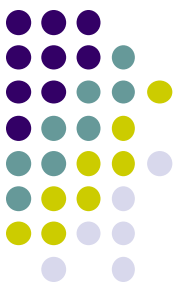


设计原则 2 - 数据驱动原则（2/3）

- 在数据仓库设计中，由于其所有数据均应建立在已有的数据库基础上，即是从已经存在于操作型环境中的数据出发进行数据仓库的设计，这种设计方法被称为“数据驱动”方法
 - 从已有的数据库系统出发
 - 按照分析领域对数据及数据间的联系重新考察
 - 利用数据模型有效识别原有数据库中的数据和数据仓库中主题的数据的“共同性”

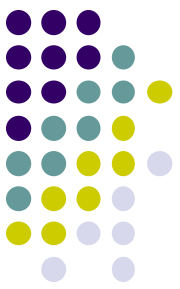
设计原则 2 - 数据驱动原则 (3/3)





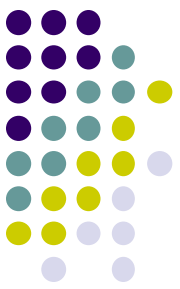
设计原则 3 - 原型法设计原则（1/3）

- 不断变化的分析需求
 - 数据仓库系统的原始需求不明确，且不断变化与增加，开发者最初并不能确切了解到用户的明确而详细的需求，用户所能提供的无非是需求的大方向或部分需求，更不能较准确地预见到以后的需求
 - 因此，采用原型法来进行数据仓库的开发是比较合适的，即从构建系统的基本框架着手，不断丰富与完善整个系统



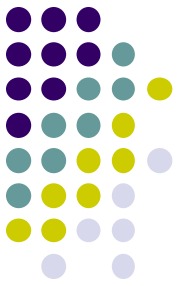
设计原则 3 - 原型法设计原则（2/3）

- 逐步求精的设计过程
 - 数据仓库用户的需求是在设计过程中不断细化明确的。同时，数据仓库系统的开发也是一个经过不断循环、反馈而使系统不断增长与完善的过程
 - 在数据仓库开发的整个过程中，自始至终要求决策人员和开发者的共同参与和密切合作，不做或尽量少做无效工作与重复工作



设计原则 3 - 原型法设计原则（3/3）

- 启发方式
 - 一个迭代的开发依赖于在上一个迭代所获得的结果
 - 首先载入一部分数据供**DSS**分析员使用和查看
 - 然后根据最终用户的反馈，修改数据和/或添加其他数据。这种反馈过程贯穿于数据仓库的整个开发生命周期之中
- 需求预测仍然是十分重要的。实际情况通常介于启发和预测两者之间



从操作型数据开始（1/7）

- 将数据从操作型环境移入到数据仓库环境不是简单的抽取

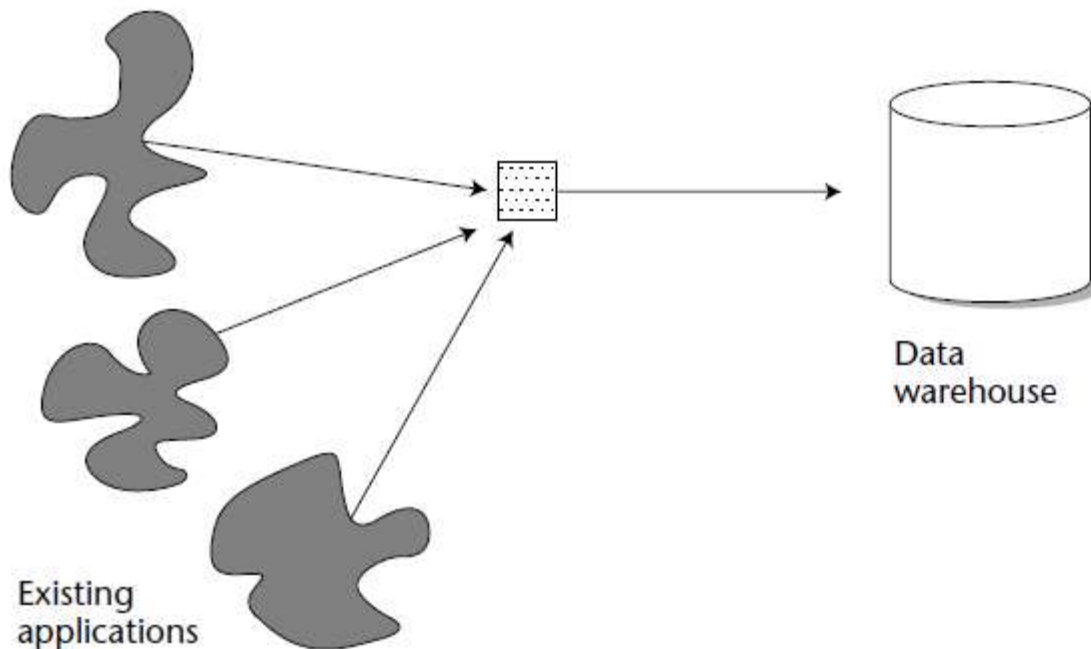
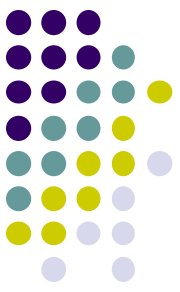


Figure 3-1 Moving from the operational to the data warehouse environment is not as simple as mere extraction.



从操作型数据开始（2/7）

- 通常现有的操作型系统中的数据缺乏集成，建立现有的操作型应用时，根本没有考虑过以后可能存在的集成问题。每一个应用都拥有自己特殊的需求，可能会出现下列情况：
 - 一些相同的数据以不同的名字出现在各个地方
 - 一些不同的数据在不同的地方以相同的方式标注
 - 这里的数据不在其他地方出现
 - 一些数据用相同的名字存在不同的地方，却使用不同的度量单位

从操作型数据开始（3/7）

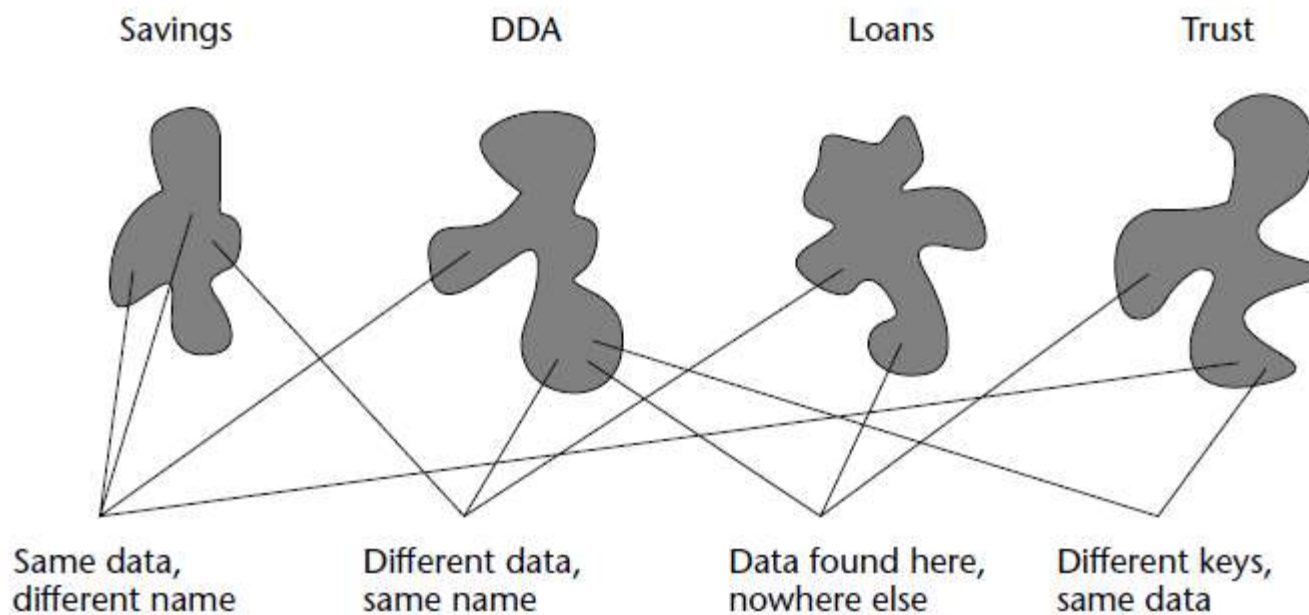
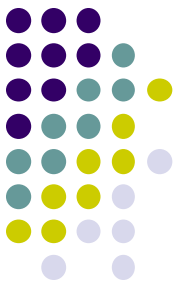


Figure 3-2 Data across the different applications is severely unintegrated.



从操作型数据开始（4/7）

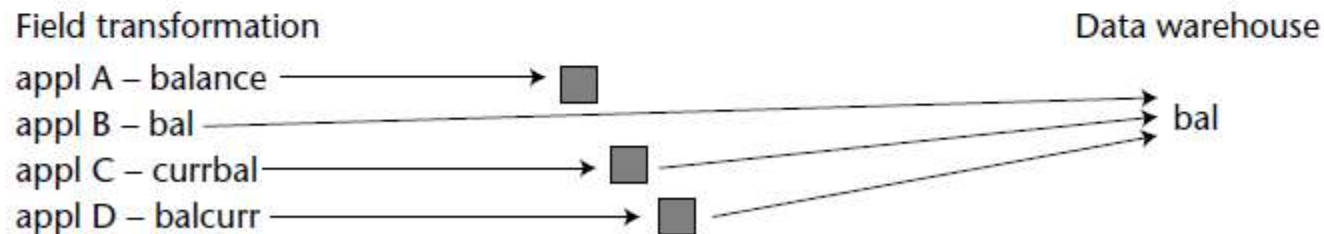
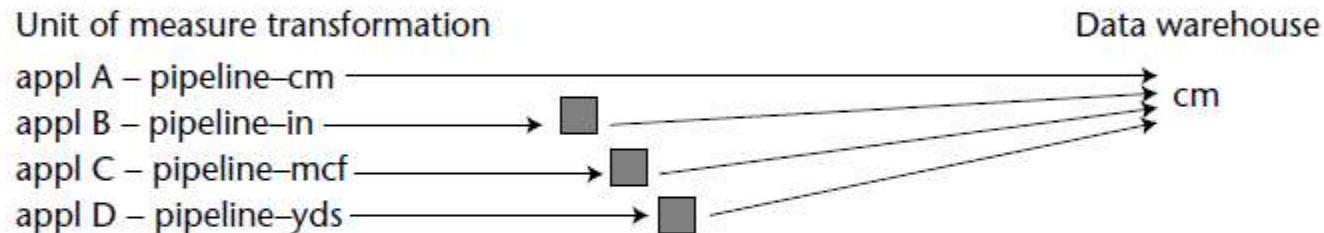
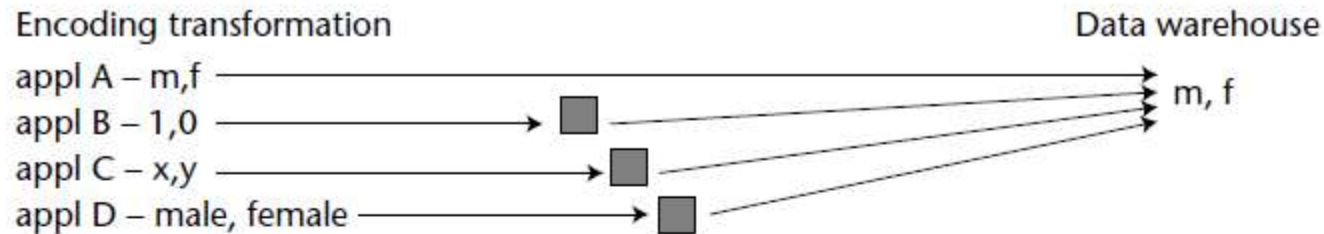
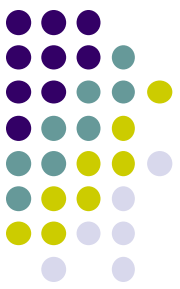


Figure 3-3 To properly move data from the existing systems environment to the data warehouse environment, it must be integrated.



从操作型数据开始（5/7）

- 从操作型环境到数据仓库有三种装载工作要做：
 - 装载档案数据
 - 装载在操作型系统中目前已有的数据
 - 将自数据库上次刷新以来在操作型环境中不断发生的变化（更新）从操作型环境中装载到数据仓库中
 - 时间戳
 - DELTA文件
 - 建立映像文件
 - 日志文件
- 需要考虑存取操作型数据的效率

从操作型数据开始（6/7）

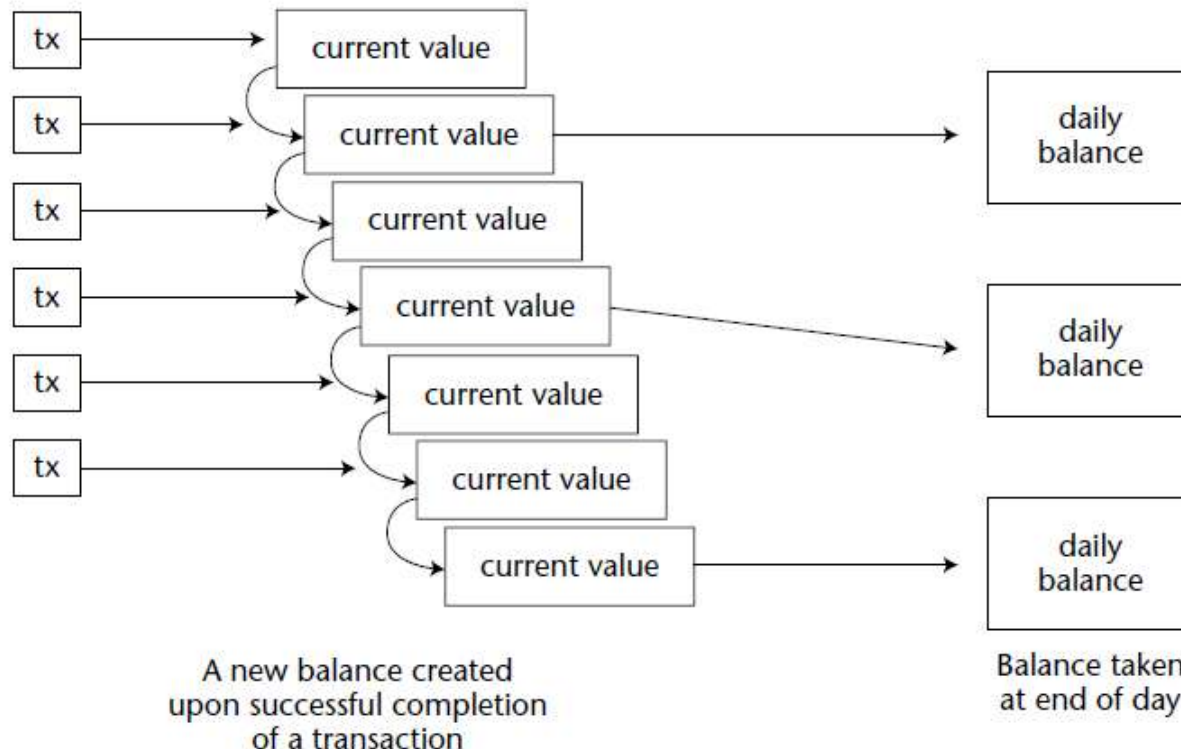
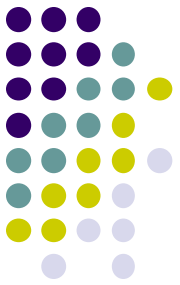
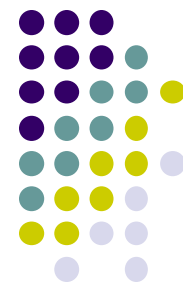


Figure 3-5 A shift in the time-basis is required as data is moved over from the operational to the data warehouse environment.

从操作型数据开始（7/7）



- 数据量的管理
 - 数据集成
 - 多粒度
 - 数据转储

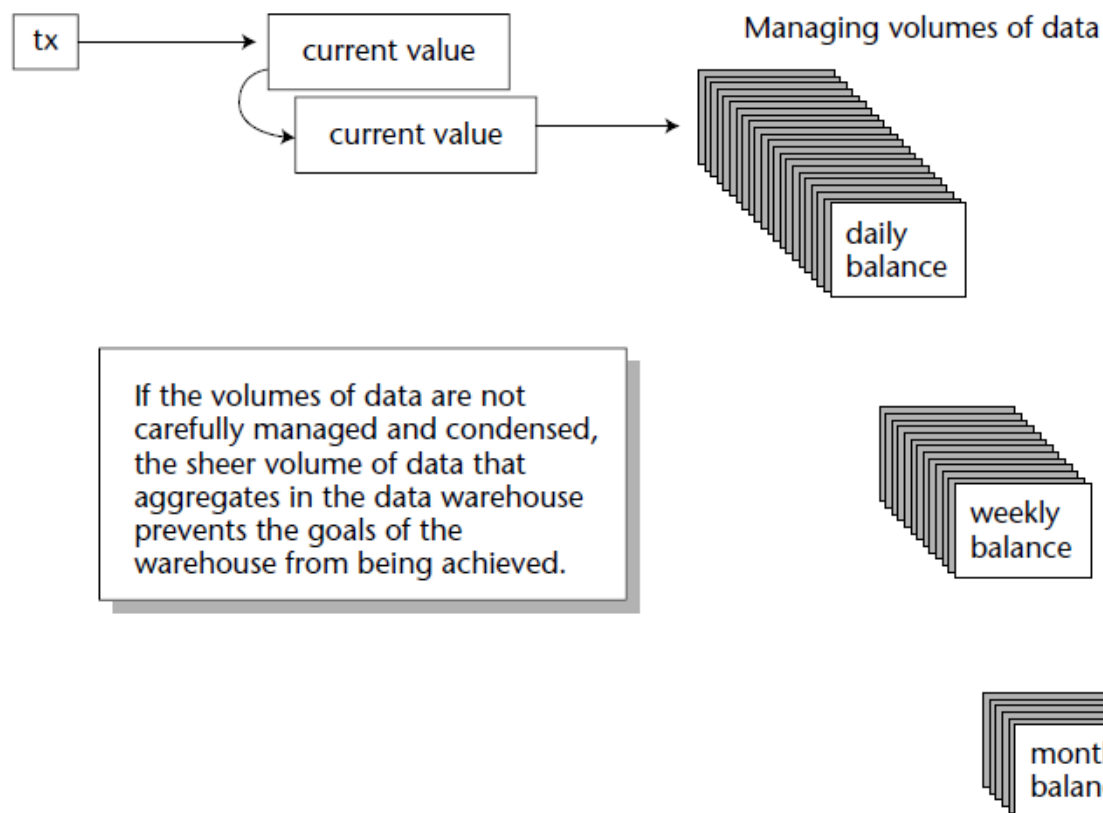


Figure 3-6 Condensation of data is a vital factor in the managing of warehouse data.

过程/数据模型与设计环境



- 过程模型（仅仅适合于操作型环境）

- 功能分解
- 第零层上下文图表
- 数据流图
- 结构图表
- 状态转换图
- HIPO 图
- 伪代码

- 数据模型（既适合操作型环境，也适合于分析型环境）

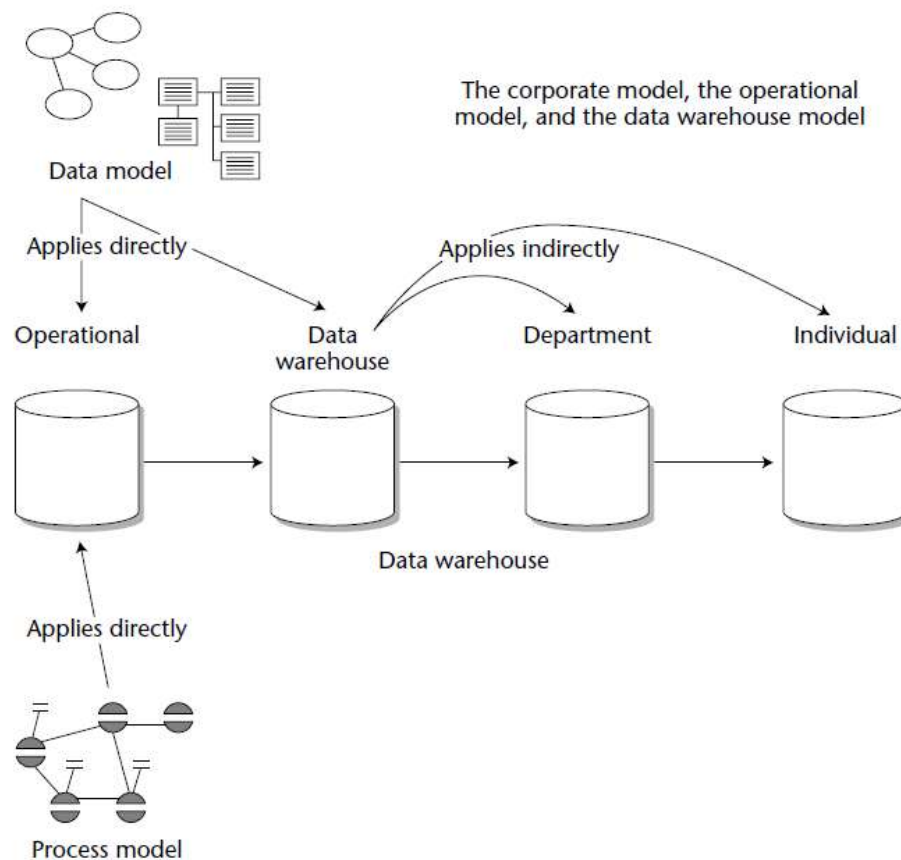
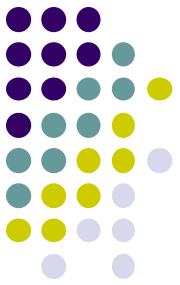
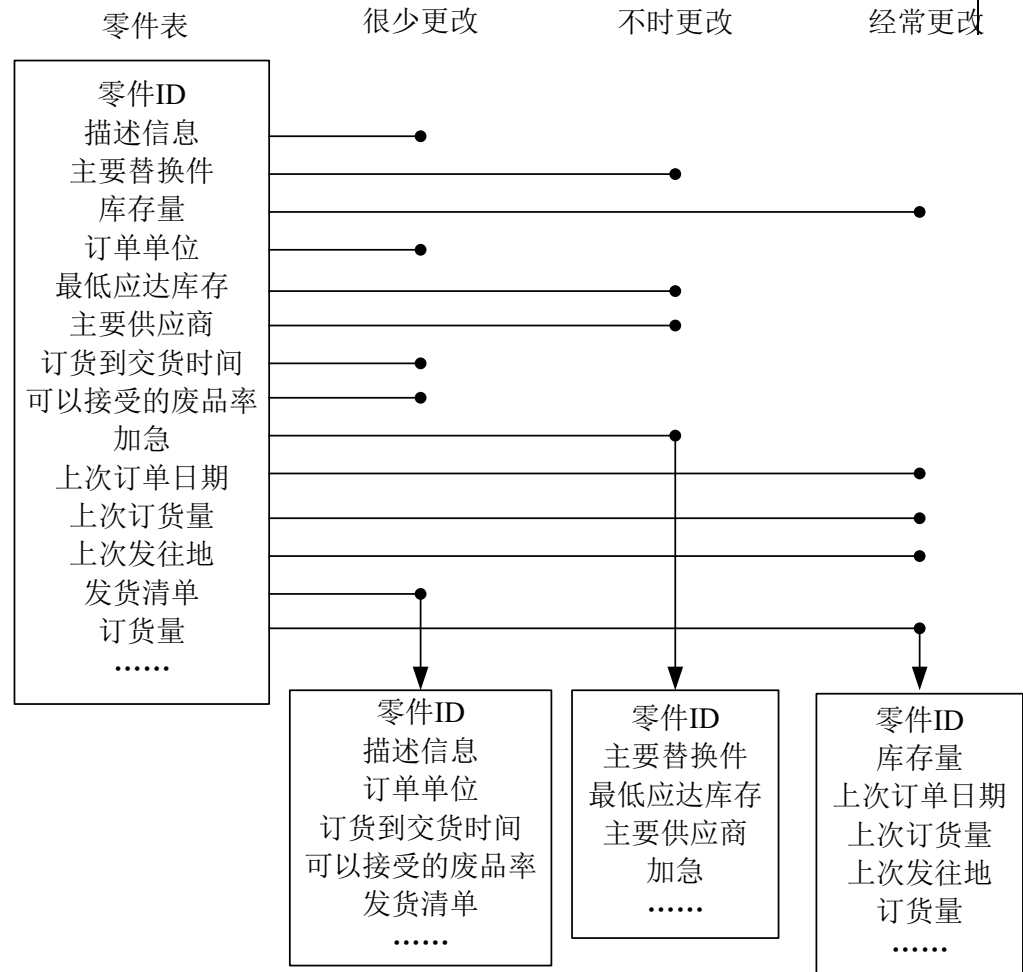


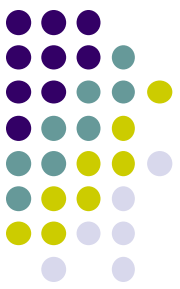
Figure 3-7 How the different types of models apply to the architected environment.



数据模型的性质

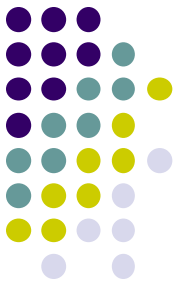
- 稳定性分析，根据各个数据属性的变化特性将这些属性分组
- 根据稳定程度讨论逻辑优化、物理优化及ETL设计等





数据模型与迭代开发（1/2）

- 迭代开发的重要性
 - 业界成功的记录强烈地建议这样做
 - 用户在第一遍迭代开发完成以前不能清晰的提出需求
 - 只有实际结果切实而且明确时，管理部门才会作出充分的承诺。因此，必须能很快地见到可见结果
- 数据模型在每一遍开发中都起着路标的作用，在开发结束时，所有遍次的开发结果融合在一起
 - 当不同遍次的开发是基于不同的数据模型或没有数据模型时，会产生众多的重复工作，且缺乏一致性



数据模型与迭代开发（2/2）

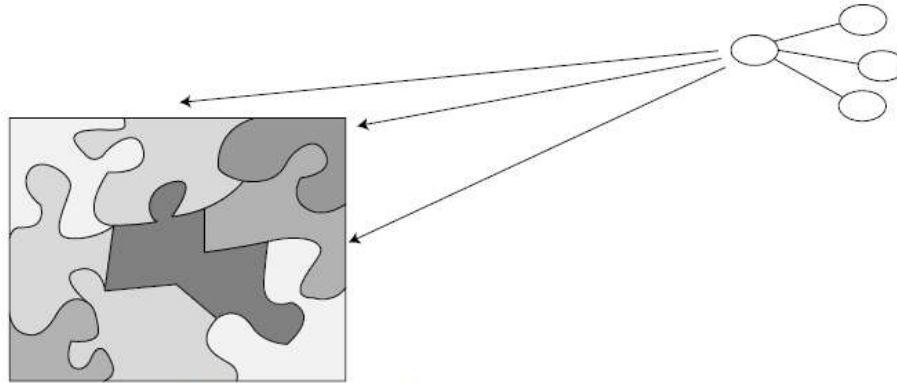


Figure 3-24 At the end of the development effort, all the iterations fit together.

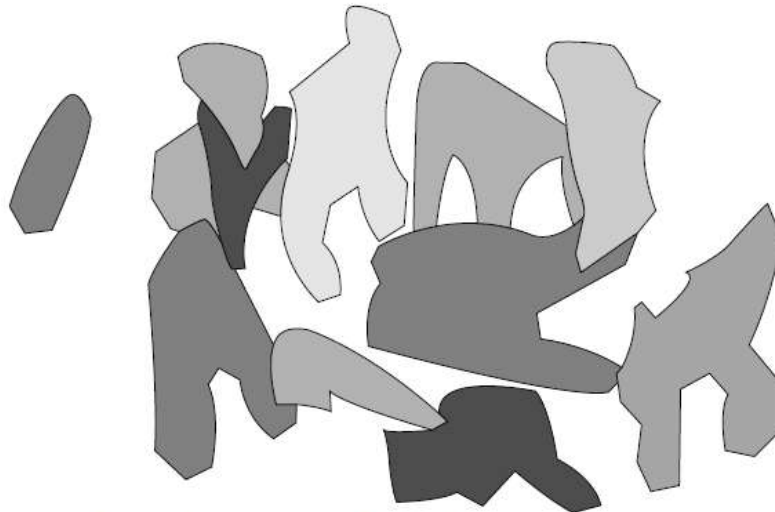


Figure 3-25 When there is no data model, the iterations do not form a cohesive pattern. There is much overlap and lack of uniformity.

数据仓库设计的三级数据模型 (1/2)

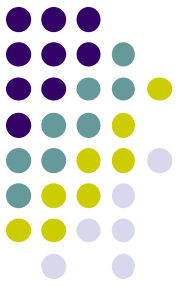


- 概念模型
 - 为一定目标设计系统、收集信息而服务的概念型工具，是客观世界到机器世界的一个中间层次
 - E-R法
- 逻辑模型
 - 描述了数据仓库的主题的逻辑实现
 - 关系模型
- 物理模型
 - 逻辑模型在数据仓库中的实现

数据仓库设计的三级数据模型 (2/2)

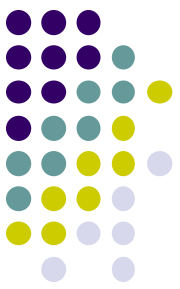


- 高级模型
 - 对数据的抽象程度最大
 - E-R图
- 中级模型
 - 数据项 (dis-data item set)
 - dis是E-R图的细分，大体上E-R图中的每个实体对应一个dis
- 低级模型
 - 物理数据模型



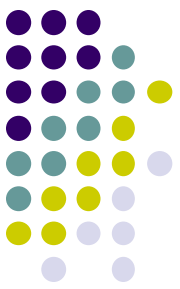
数据仓库的设计步骤

- 数据仓库设计大致有如下几个步骤:
 1. 系统规划
 - 明确主题
 - 技术准备
 2. 概念设计
 3. 逻辑设计
 4. 物理设计
 5. 数据仓库生成
 6. 数据仓库的运行与维护



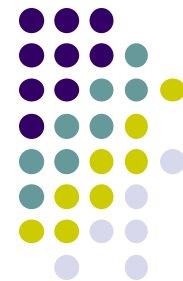
系统规划 - 明确主题

- 在数据仓库设计的开始，首先要做的事是有关分析人员需要确定具体领域的分析对象，这个对象就是主题。
- 主题是一种较高层次的抽象，对它的认识与表示是一个逐步完善的过程。因此，在开始时不妨先确定一个初步的主题概念以利于设计工作的开始，此后随着设计工作的进一步开展，再逐步扩充与完善（原型设计法）

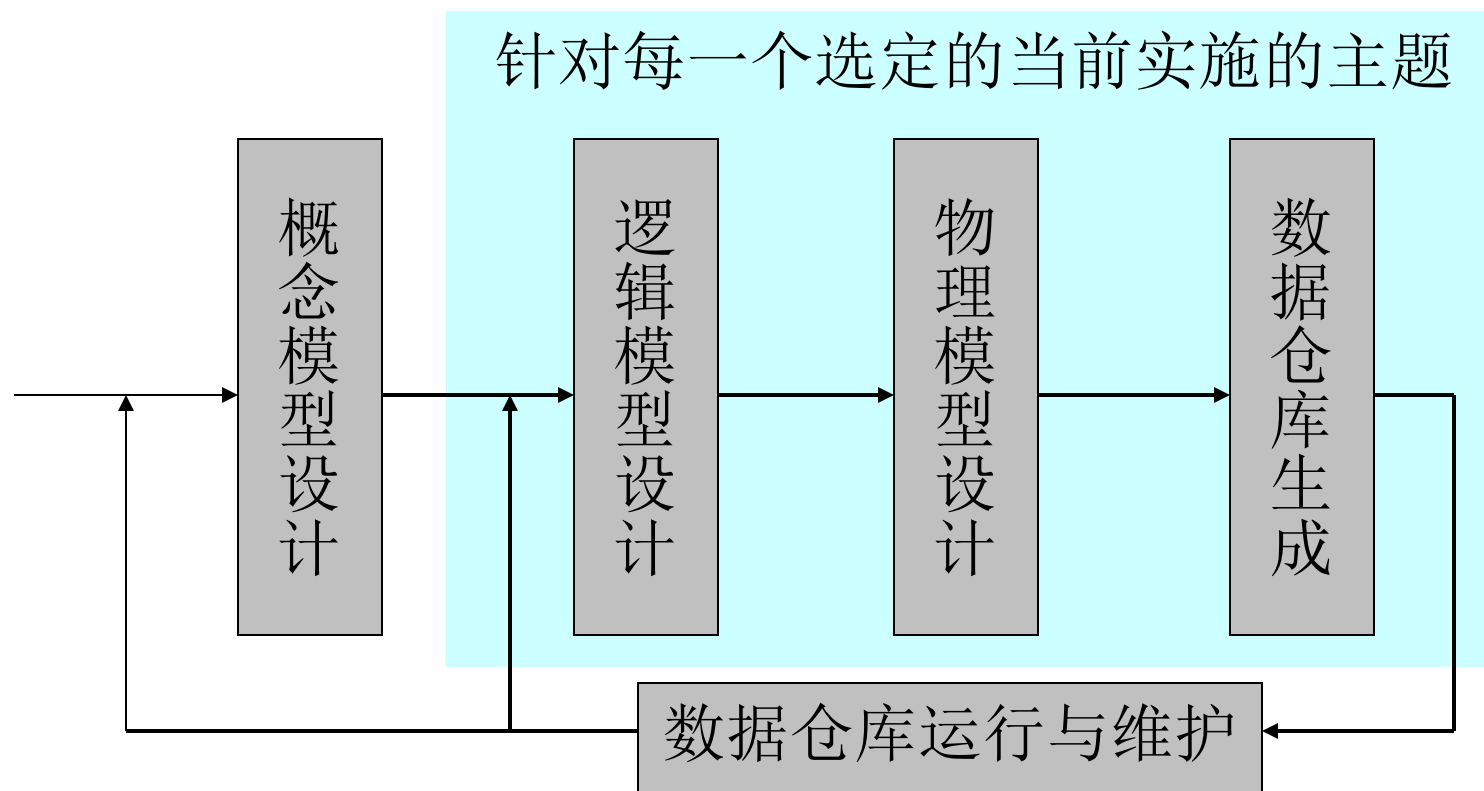


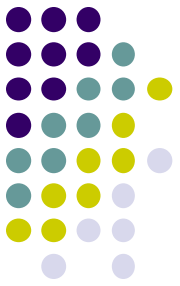
系统规划 - 技术准备

- 准备具体的技术要求和物理实现环境，包括：
 - 技术评估
 - 其内容包括数据仓库的性能指标，如：
 - 数据存取能力（包括管理海量数据的能力）
 - 模型重组能力
 - 数据装载能力
 - 技术环境准备
 - 在评估基础上提出数据仓库的软硬件平台要求，包括计算机、网络结构、操作系统、数据库及数据仓库软件的选购要求等



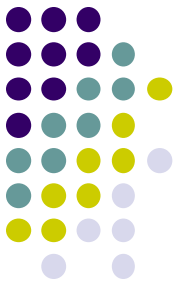
数据仓库的设计/生成和运维





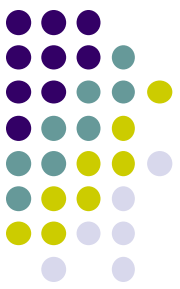
概念模型设计

- 确定系统边界
- 确定主要的主题及其内容
- OLAP等分析应用的设计



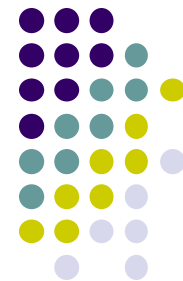
确定系统边界

- 要做的决策类型有哪些？
- 决策者感兴趣的是什么问题？
- 这些问题需要什么样的信息？
- 要得到这些信息需要包含哪些数据源？

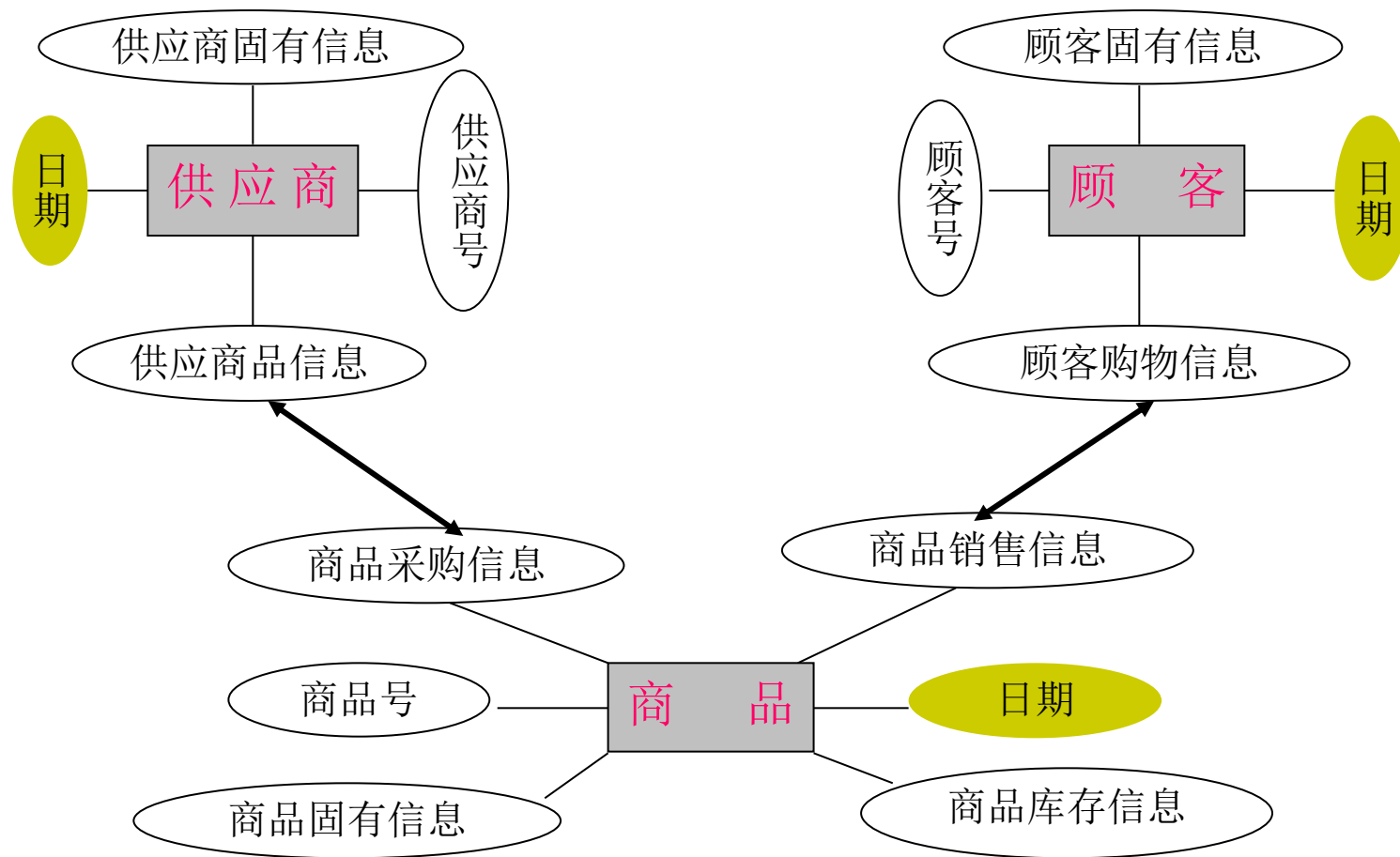


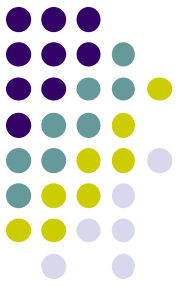
确定主要的主题及其内容

- 确定主要的主题
 - 明确数据仓库的分析对象，然后对每个主题的内容进行较详细的描述，包括：
 - 确定主题及其属性信息
 - 描述每个属性的取值情况
 - 固定不变的
 - 半固定的
 - 经常变化的
 - 确定主题的公共码键
 - 主题之间的关系：主题间联系及其属性
- 在确定上述内容后，就可以用传统的实体联系模型（E-R模型）来表示数据仓库的概念数据模型



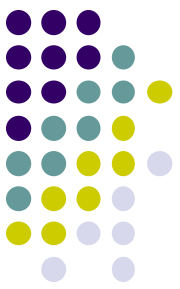
商品、顾客和供应商之间的E-R图





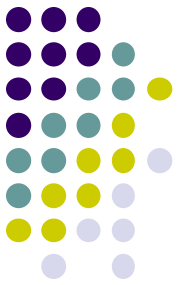
逻辑模型设计

- 将E-R图转换成关系数据库的二维表
- 定义数据源和数据抽取规则
- 在逻辑模型的设计过程中，需要考虑以下一些问题：
 - 适当的粒度划分
 - 合理的数据分割策略
 - 定义合适的数据来源
 -



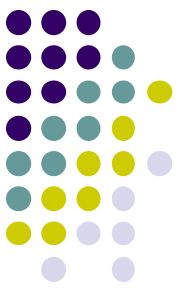
逻辑模型设计 – 粒度划分

- 在设计过程中需要考虑数据仓库中数据粒度的划分原则，即数据单元的详细程度和级别
 - 数据越详细，粒度越小，级别就越低
 - 数据综合度越高，粒度越大，级别就越高
- 一般将数据划分为：详细数据、轻度总结、高度总结三种粒度，或者采用更多级的粒度划分方法。例如：
 - 根据时间跨度进行的统计有：天，周，月，季度，年
 - 对于不适合进行统计的属性值，可以采样获取数据
- 粒度的划分将直接影响到数据仓库中的数据量以及所适合的查询类型，粒度划分是否适当是影响数据仓库性能的一个重要方面



商品销售 – “时间”属性的粒度设计

- 商品固有信息：
 - 商品表（商品号，商品名，类型，颜色，...）/* 细节数据 */
- 商品销售信息：
 - /* 细节数据 */
 - 销售表（商品号，顾客号，销售日期，售价，销售量，...）
 - /* 以“天”为时间统计单位的综合数据 */
 - 销售表d1（时间属性_日，商品属性，...，销售总量）
 - /* 以“周”为时间统计单位的综合数据 */
 - 销售表d2（时间属性_周，商品属性，...，销售总量）
 - /* 以“月”为时间统计单位的综合数据 */
 - 销售表d3（时间属性_月，商品属性，...，销售总量）
 - /* 以“季度”为时间统计单位的综合数据 */
 - 销售表d4（时间属性_季度，商品属性，...，销售总量）
 - /* 以“年”为时间统计单位的综合数据 */
 - 销售表d5（时间属性_年，商品属性，...，销售总量）
 -



商品销售 – “商品” 属性的粒度设计

- 商品销售信息（续）

/* 细节数据 */

- 销售表（商品号，顾客号，销售日期，售价，销售量，...）

/* 以具体“商品”为商品属性的统计单位的综合数据 */

- 销售表p1（时间属性_X，商品属性_商品号，...，销售总量）

/* 以“小类”为商品属性的统计单位的综合数据 */

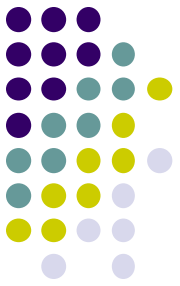
- 销售表p2（时间属性_X，商品属性_小类，...，销售总量）

/* 以“大类”为商品属性的统计单位的综合数据 */

- 销售表p3（时间属性_X，商品属性_大类，...，销售总量）

.....

- 将时间、商品等属性综合起来考虑，上述的统计表有很多！



商品库存 – “时间”属性的粒度设计

- 商品库存信息：

- /* 细节数据 */

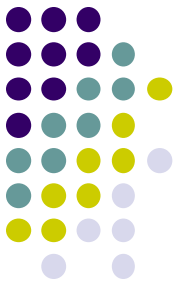
- 库存表（商品号，库房号，库存量，日期， ...）

- /* 样本数据 */

- 库存表1（商品号，库房号，库存量，取样时间_周， ...）
 - 库存表2（商品号，库房号，库存量，取样时间_月， ...）
 - 库存表3（商品号，库房号，库存量，取样时间_季度， ...）
 - 库存表4（商品号，库房号，库存量，取样时间_年， ...）

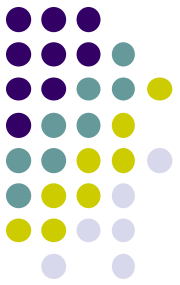
-

- 数据综合的方式与前面的商品销售主题不同



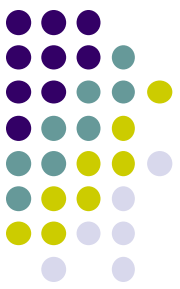
用于商场管理者的数据仓库（1/2）

主题名	公共键	属性信息
商品	商品号	●固有信息：商品号，商品名，类别，颜色等 ●采购信息：商品号，供应商号，供应价，供应日期，供应量等 ●销售信息：商品号，顾客号，售价，销售日期，销售量等 ●库存信息：商品号，库房号，库存量，日期等



用于商场管理者的数据仓库（2/2）

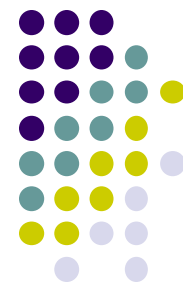
主题名	公共键	属性信息
供应商	供应商号	● 固有信息：供应商号，供应商名，地址，电话，供应商类型等 ● 供应商品信息：供应商号，商品号，供应价，供应日期，供应量等
顾客	顾客号	● 固有信息：顾客号，姓名，性别，年龄，文化程度，住址，电话等 ● 购物信息：顾客号，商品号，售价，购买日期，购买量等



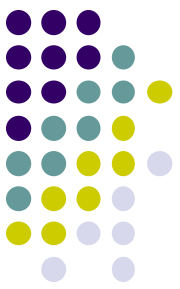
逻辑模型设计 – 数据分割

- 数据的分割是指把逻辑上是统一整体的数据分割成较小的、可以独立管理的数据单元进行存储（关系），以便于重构、重组和恢复，以提高创建索引和顺序扫描的效率
- 选择数据分割的因素有：
 - 数据量的大小
 - 数据分析处理的对象（主题）
 - 简单易行的数据分割标准
 - 数据粒度的划分策略
- 通常采用“时间”属性作为数据分割的依据
- 数据分割技术类似于数据库中的数据分片技术，其目的是为了提高数据仓库的性能

逻辑模型设计 – 定义数据来源及其抽取规则

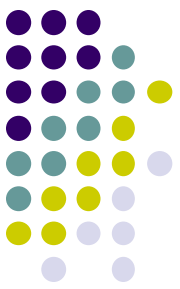


主题名	属性名	数据源系统	源表名	源属性名
商品	商品号	库存子系统	商品	商品号
商品	商品名	库存子系统	商品	商品名
商品	类别	采购子系统	商品	类别
.....				



物理模型设计

- 在逻辑模型设计基础上确定数据的存储结构、确定索引策略、确定存储分配及数据存放位置等与物理有关的内容，物理模型设计的具体方法与数据库设计中的大致相似。其目的是为了提_高数据仓库系统的访问性能
- 常用的一些技术有：
 - 合并表
 - 建立数据序列
 - 引入冗余
 - 表的物理分割
 - 生成导出数据
 - 建立广义索引
- 规范化/反规范化



物理模型设计 – 合并表（1/2）

- 在常见的一些分析处理操作中，可能需要执行多表连接操作。为了节省I/O开销，可以把这些表中的记录混合存放在一起，以减低表的连接操作的代价
- 合并表技术与传统关系数据库中的集簇(Clustering)技术类似

物理模型设计 – 合并表 (2/2)

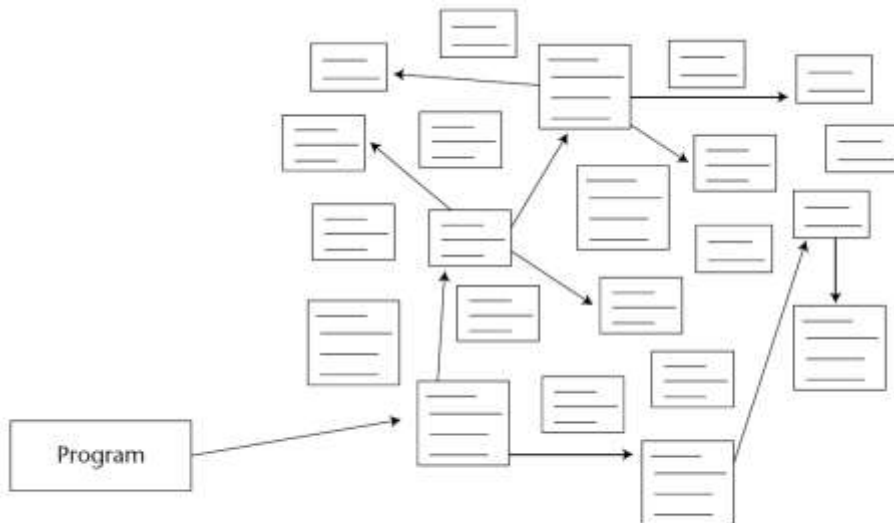
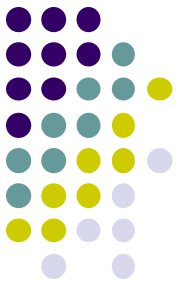


Figure 3-26 When there are many tables, much I/O is required for dynamic interconnectability.

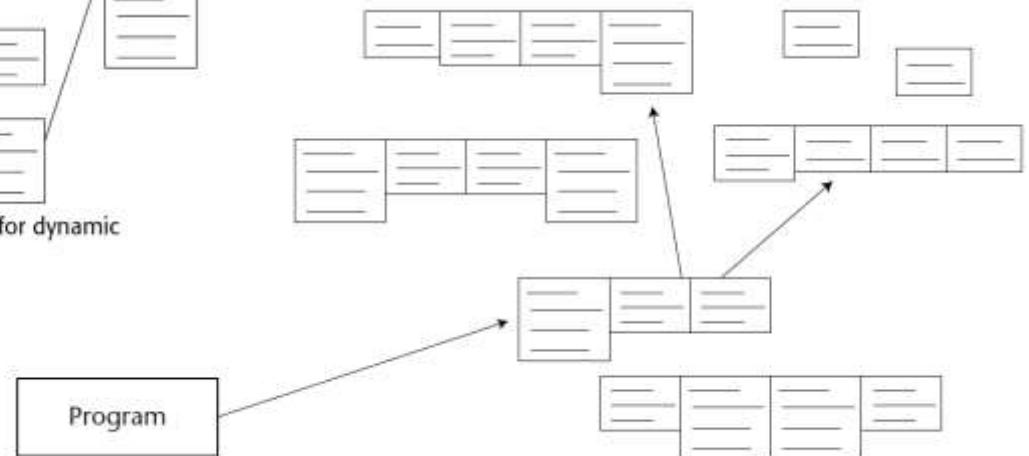
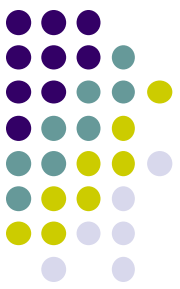


Figure 3-27 When tables are physically merged, much less I/O is required.



物理模型设计 – 数据序列（1/2）

- 考虑创建一个数据序列，这样如果数据存放在一行中，那么一次I/O就足以检索到了
- 通常当数列中值的数量稳定、数据是按顺序访问的、数据的创建与修改在统计上是以非常有规律的方式进行等条件都满足时，创建一个数据序列才是有意义的

物理模型设计 – 数据序列 (2/2)

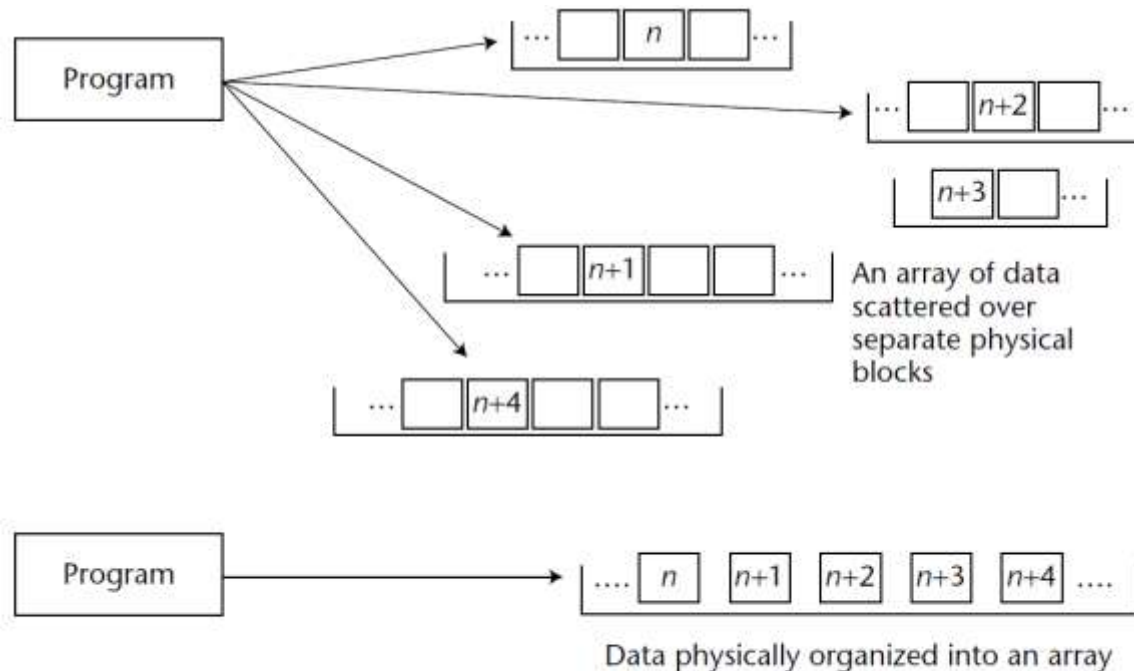
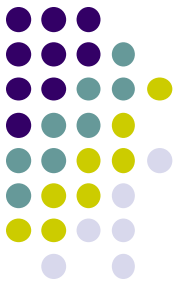
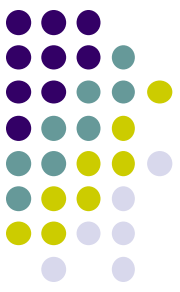
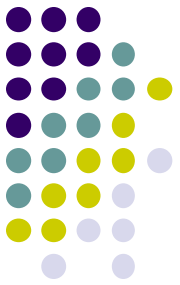


Figure 3-28 Under the right circumstances, creating arrays of data can save considerable resources.

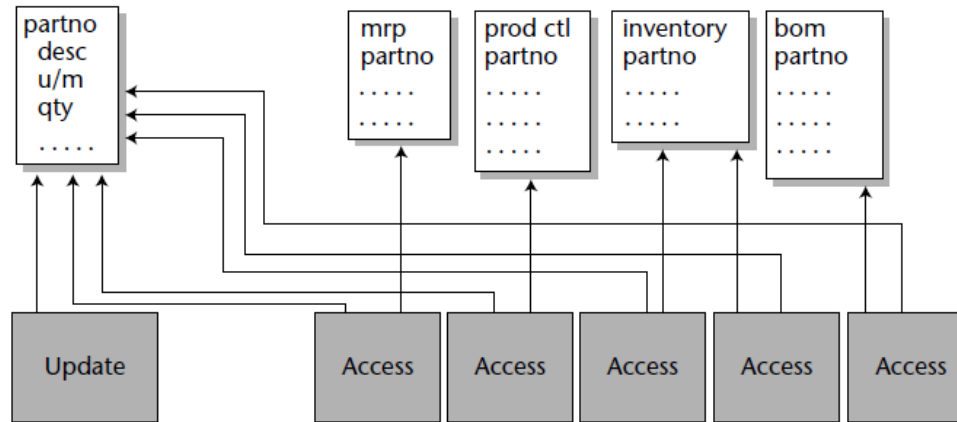


物理模型设计 – 引入冗余（1/2）

- 在面向某个主题的分析过程中，通常需要访问不同表中的多个属性，而每个属性又可能参与多个不同主题的分析过程。因此可以通过修改关系模式把某些属性复制到多个不同的主题表中去，从而减少一次分析过程需要访问的表的数量
- 采用该种数据组织方法会带来大量的数据冗余存储，数据仓库系统必须保证这些冗余数据的一致性。由于数据仓库中的数据是稳定的，很少执行更新操作，不会因此带来过高的数据更新的代价，但可以有效地提高数据仓库系统的性能



物理模型设计 – 引入冗余 (2/2)



Description is nonredundant and is used frequently, but is seldom updated.

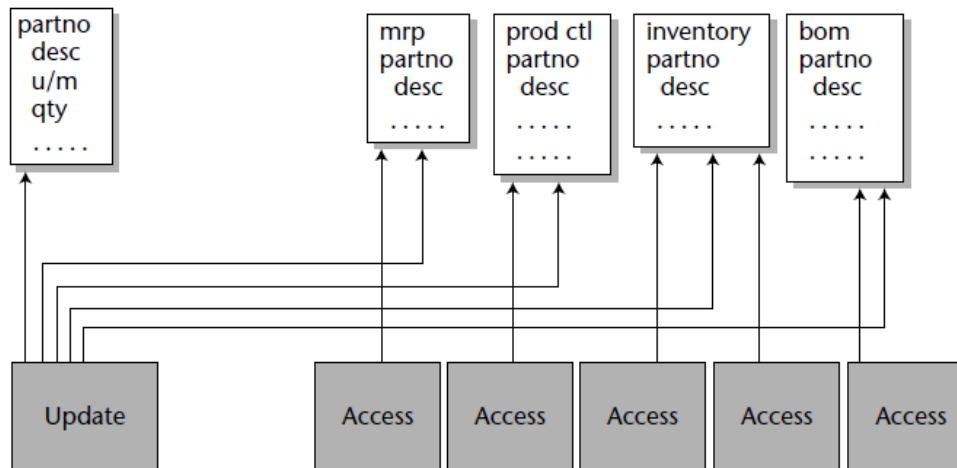
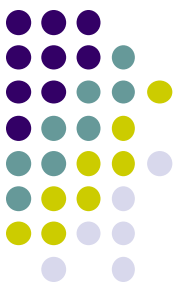
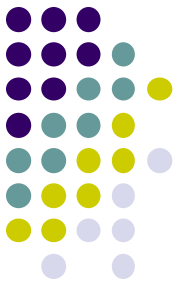


Figure 3-29 Description is redundantly spread over the many places it is used. It must be updated in many places when it changes, but it seldom, if ever, does.



物理模型设计 – 表的物理分割（1/2）

- 类似于在逻辑设计阶段的数据分割
- 可以根据表中每个属性数据的访问频率和稳定性程度对表的存储结构进行分割
 - 对于访问频率较高的属性，可以单独考虑其物理存储组织，以便选择合适的索引策略和特定的物理组织方式
 - 对于需要频繁更新的属性，也可以单独组织其物理存储，避免因数据更新而带来的空间重组、重构等工作



物理模型设计 – 表的物理分割（2/2）

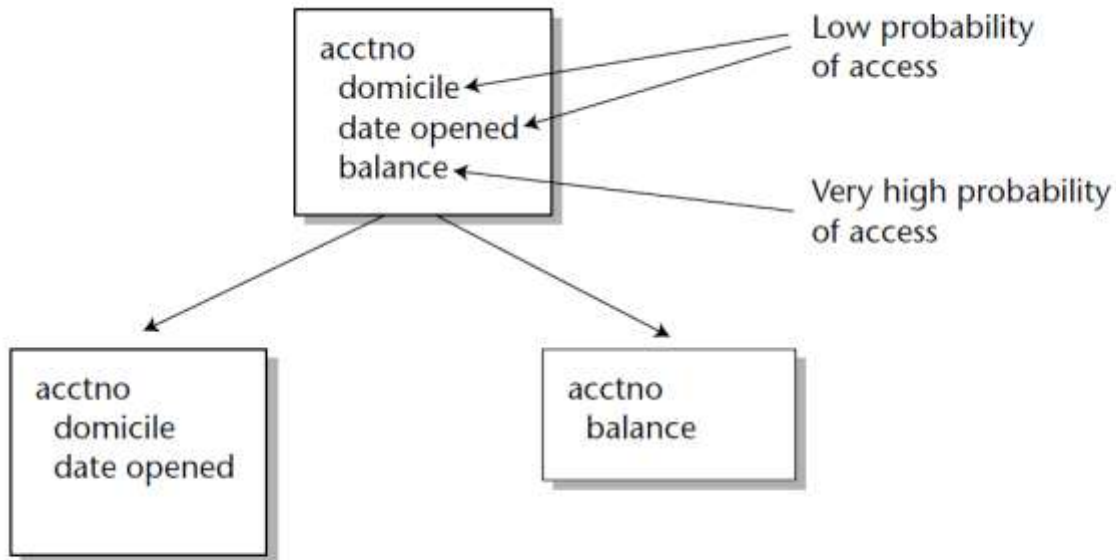
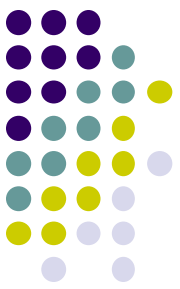
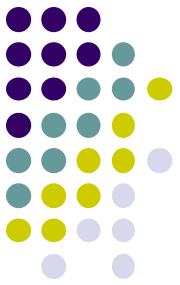


Figure 3-30 Further separation of data based on a wide disparity in the probability of access.



物理模型设计 – 生成导出数据 (1/2)

- 在原始、细节数据的基础上进行一些统计和计算，生成导出数据，并保存在数据仓库中
- 优点
 - 避免在分析过程中执行过多的统计或计算操作，减少输入/出的次数，提高分析操作的性能
 - 避免了不同用户进行重复统计操作可能产生的偏差



物理模型设计 – 生成导出数据 (2/2)

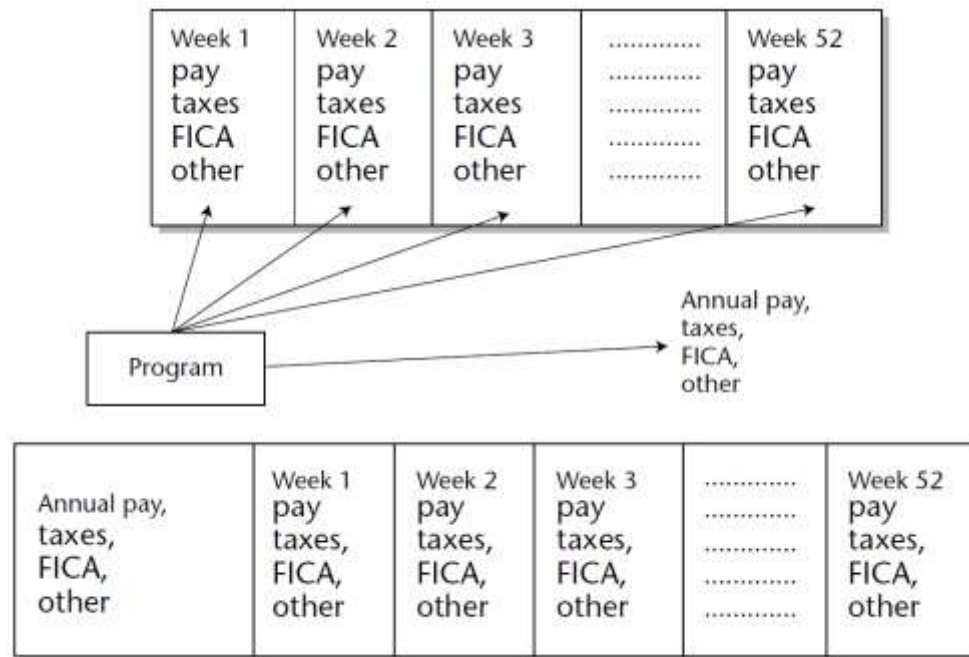
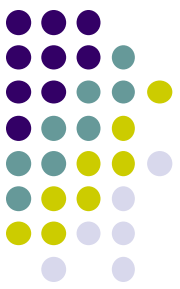


Figure 3-31 Derived data, calculated once, then forever available.



物理模型设计 – 建立广义索引（1/2）

- 用于记录数据仓库中数据与“最”有关的统计结果的索引被称为“广义索引”。如：
 - 当月销售额最高的商店？
 - 当月销售情况最差的商品？
 -
- 这样的广义索引的数据量是非常小的，可以在每次进行数据仓库数据加载工作时生成或刷新这样的广义索引。用户可以从已经建立的广义索引里直接获取这些统计信息，而不必对整个数据仓库进行扫描

物理模型设计 – 建立广义索引 (2/2)

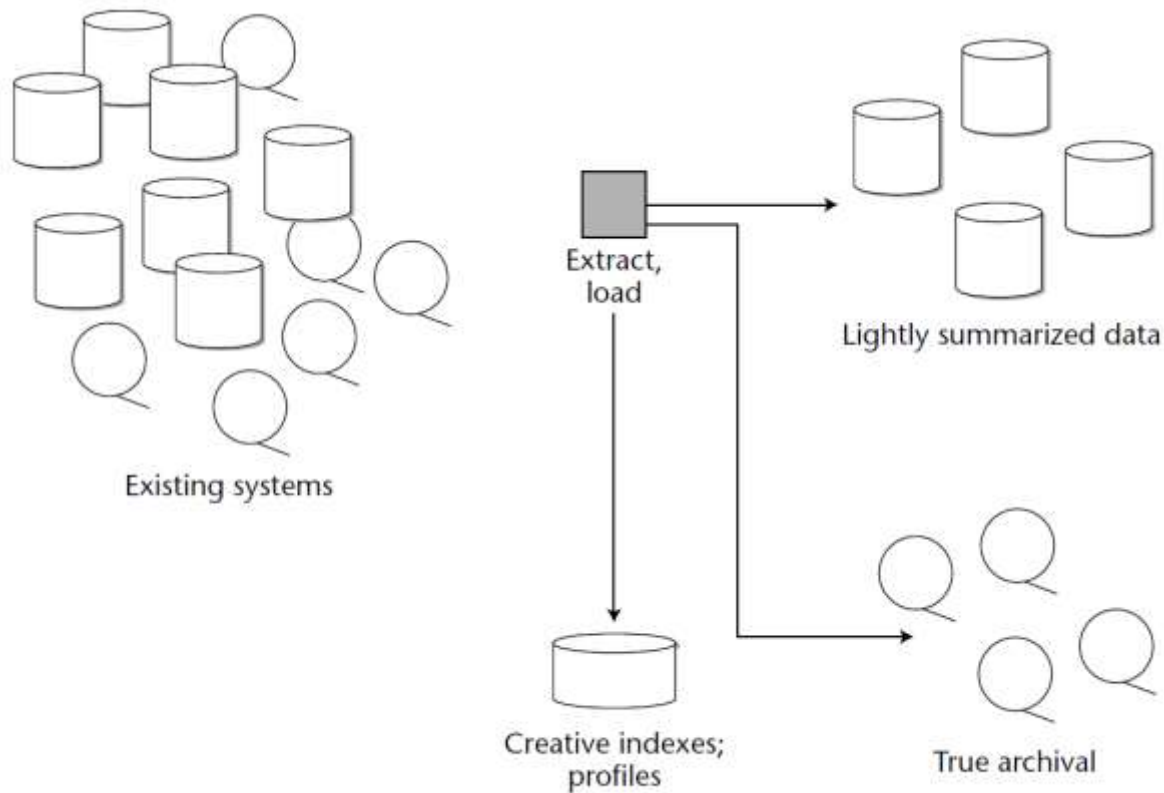
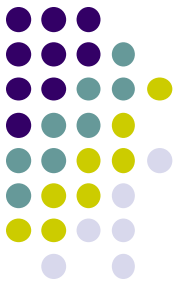
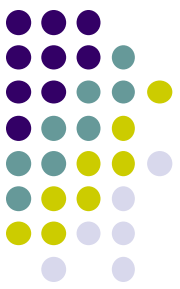
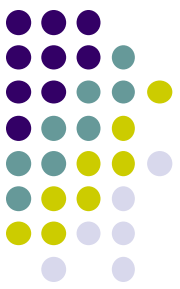


Figure 3-32 Examples of creative indexes.



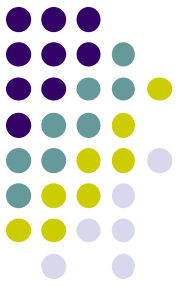
数据仓库生成

- 建立数据模式
 - 根据逻辑设计与物理设计的设计结果建立数据仓库的数据模式
- 编制数据抽取程序
 - 根据数据仓库元数据中的定义信息，编制抽取程序，将数据源中的数据作加工以形成数据仓库中的数据
- 数据加载
 - 将数据源中的数据，通过数据抽取程序加载到数据仓库的数据模式中去



数据仓库的使用与维护

- 在数据仓库建立后，就可以建立分析、决策型的应用系统
- 在应用系统的使用过程中不断加深理解，改进主题，依照原型法的思想使系统更趋完善
- 在系统的运行过程中，随着数据源中数据的不断变化，需要通过数据刷新操作来维护数据仓库中数据的一致性



数据仓库的生命周期

